Agentic Security Platform

Cursor AI Plan Mode Project Summary

Governed multi-agent security on GKE

This document is the filled Plan Mode record for the live lab. It follows the Cursor AI Plan Mode prompt template (30 June 2026): clarification first, then research, then an eight-part plan, then review, then implementation only after approval.

Google Cloud | GKE Dataplane V2 | Vertex AI / Gemini | OPA | Cloud KMS | MCP | GitHub Actions

Updated 18 August 2026. Scripts `01`–`15`. Terraform `01`–`25`. Fail-closed defaults remain.

Field	Value
Project type	Student portfolio lab with production-minded architecture and security controls
Current status	Live cluster `vertex-agent-lab` in `us-central1-c`; remediation remains disabled
Complexity	9/10
Safety posture	Fail-closed; Slack/Jira reporting off by default
Prepared by	T.I.Q.S.
Audience	Technical review, recruiter discussion, and guided implementation
Template source	Cursor AI Plan Mode prompt template, 30 June 2026

Plan Mode expectations

During Plan Mode:

1. Focus on strategy and design, not implementation details.
2. Ask specific, technical questions.
3. Reference existing code patterns when relevant.
4. Identify potential problems before they occur.
5. Provide clear reasoning for each decision.
6. Wait for approval before writing any code.

Plan Mode rule: no live remediation, production deployment, or irreversible infrastructure change until the plan is reviewed, the required tests pass, and an authorized reviewer approves the action.

Network architecture

Canonical diagrams in this folder:

- `agentic-network.svg` / `agentic-network.png`
- `agentic-ai-workflow.svg`
- `architecture.md`, `namespace-architecture.md`, `pod-architecture.md`, `mcp-architecture.md`

Gemini never authorizes. Detect/analyze sits on the left of the cluster. Govern/execute sits on the right. Pub/Sub is the only hop between agents.

Document roadmap

Section	Template mapping
Phase 1	Clarification — questions only, then resolved answers
Phase 2	Research — files, dependencies, and patterns
Phase 3	Plan generation — the eight required Markdown sections
Phase 4	Review — approval gate before code or controlled execution
Phase 5	Implementation — production-ready sequence after approval
Appendix A	Major component summary
Appendix B	Complete reusable Plan Mode prompt (direct copy)
Appendix C	How to use the template, modular prompts, and tips
Appendix D	Lab changes through 18 August 2026
Appendix E	Real-world relevance, enterprise outlook, and elevator speeches

---

Phase 1: Clarification

ONLY ask questions. Do NOT write any code yet.

Analyze the request and ask clarifying questions about:

- Project scope and requirements
- Technologies and stack involved
- Existing codebase and constraints
- Specific goals and success criteria
- Any blockers or dependencies

Clarifying questions asked

1. **Group and platform name.** Is this a generic agentic demo, or a governed security platform on a named GKE cluster?
2. **Cloud project, region, and zone.** Which Google Cloud project hosts the lab, and which zone should the cluster use after N2 stockouts?
3. **Stack.** Terraform, GKE Dataplane V2, Pub/Sub, Vertex AI / Gemini, OPA, Cloud KMS, Firestore, BigQuery, MCP, GitHub Actions WIF — confirm what is in scope versus out of scope (Datadog, Kyverno, Redis, Calico).
4. **Target workload.** Which namespace and Deployment may remediation ever touch? Is there a `vertex-agent` image?
5. **Identities.** Which account may administer Terraform/GKE, which Workspace user may sign approvals, and who may own the BigQuery evidence dataset under `iam.allowedPolicyMemberDomains`?
6. **Execution posture.** Should automated remediation start enabled, or remain fail-closed until a later reviewed change?
7. **Success criteria.** What must a demo prove without executing a cluster change?
8. **Blockers.** Git Bash `bq.cmd` / `\r` issues, kubectl impersonation cache, org-policy member domains, DESTROYED KMS version 1 — which of these are live constraints?
9. **State backend.** Local Terraform state or GCS?

10 **Existing infrastructure.** Clean start, or adopt retained KMS / WIF / custom roles after destroy?

Resolved answers

Clarification area	Resolved requirement
Project scope	Build a governed, event-driven security automation platform on GKE. AI assists analysis but does not independently authorize or execute infrastructure changes.
Cloud and region	Project `class-6-5-tiqs`. Region `us-central1`. Zone `us-central1-c` (moved off `us-central1-b` after N2 stockouts). Cluster `vertex-agent-lab`. Node type `e2-standard-4`. Datapath Dataplane V2, not Calico.
Core stack	Terraform `01`–`25`, GKE, Workload Identity, Pub/Sub, Vertex AI / Gemini 2.5 Flash, OPA/Rego, Cloud KMS (approval signing version **2**), Firestore, BigQuery, Secret Manager, Cloud Logging, Cloud Monitoring / Managed Prometheus, Docker, GitHub Actions WIF, Trivy, Syft, Cosign.
Out of scope	Datadog, Kyverno, Redis, Calico network policy, a `monitoring` namespace, and a `vertex-agent` image.
Primary security objective	Deterministic authorization, signed human approval, least privilege, mTLS, replay prevention, immutable releases, and durable evidence.
Target workload	`broken-app` (`app:v1`) and PostgreSQL in `app01`.
Identities	Gmail project Owner may administer the cluster and Terraform. Consumer Gmail cannot be on the BigQuery dataset ACL or `evidence-admin` IAM. Workspace signer `APPROVER_EMAIL` is in `approval_signer_members`. Dataset OWNER is `evidence-admin@class-6-5-tiqs.iam.gserviceaccount.com`.
Default execution posture	`AUTOMATION_MODE: disabled` and `EXECUTION_MODE: disabled` until explicitly changed to `controlled`.
Success definition	Findings reach an incident report; sensitive actions require valid governance and approval evidence; no bypass around MCP, mTLS, policy, or RBAC.
State storage	Remote GCS backend `gs://tiqs-kubernetes/agentic/terraform/state`.
Existing infrastructure	Not a clean start after destroy. Run `03-adopt-retained-gcp.sh` for KMS key rings, WIF pools, and custom roles that survive teardown.
Known operators issues	Git Bash (`bq.cmd`, `\r` in `gcloud` lists, `CLOUDSDK_PYTHON`); kubectl impersonation cache; org-policy IAM member domains.

Clarification checklist

- [x] Scope and in-scope services are documented.
- [x] Google Cloud project, region, zone, and cluster naming are defined.
- [x] Target Kubernetes namespaces and allowlisted workload are defined.
- [x] Fail-closed defaults are documented.
- [x] Cloud KMS reviewer identity is a Workspace user (`APPROVER_EMAIL`); key version `1` is DESTROYED; live version is `2`.
- [ ] Administrator workstation CIDR remains in ignored `terraform/terraform.tfvars`.
- [ ] Production GitHub environment reviewer must be confirmed for digest deploy.

If requirements are clear, ask 3–5 clarifying questions and proceed to plan. If requirements are vague, ask more detailed questions before planning. Suggest alternative approaches when a better solution exists.

Phase 2: Research

After the answers:

- Review the information provided.
- Identify relevant files and components needed.
- Check for potential dependencies.
- Consider best practices and patterns.

Current-state findings

Area	Finding
Architecture	Multi-stage Pub/Sub pipeline with isolated analysis, governance, approval, remediation, and reporting roles.
Identity	Dedicated Kubernetes and Google service accounts via Workload Identity. GitHub uses OIDC federation.
Governance	OPA/Rego produces `DENY`, `NO_ACTION`, `PERMIT`, or `REQUIRE_APPROVAL`.
Approval	Cloud KMS asymmetric signing binds the reviewer to request, decision, expiry, and key version `2`.
State	Firestore stores correlation windows, approval consumption, and execution idempotency.
Execution boundary	Only the Remediation Agent has an MCP client certificate. nginx MCP Gateway; MCP Server holds `app01` RBAC.
Evidence	Cloud Logging and BigQuery retain governance, approval, remediation, and MCP evidence. Query BigQuery by impersonating `evidence-admin`.
Reliability	Pub/Sub retry/DLQ, HPA, PDB, probes, resource limits, PodMonitoring.
Networking	Dataplane V2 NetworkPolicy. DNS allows kube-dns, node-local-dns, `10.112.0.10/32`, and `169.254.20.10/32`.
Supply chain	`ci.yml` and `release.yml`; Trivy, Syft, Cosign; immutable digests; scripts `13` and `14`.

Relevant files and components

Path	Responsibility
`.github/workflows/ci.yml`	PR and main lint, Python tests, Terraform validate, OPA, Conftest, Trivy
`.github/workflows/release.yml`	Signed image release; optional digest deploy
`docker/`	One build definition per runtime image
`docs/RUNBOOK.md`	Operator commands
`docs/architecture/`	Network, workflow, namespace, pod, and MCP diagrams
`manifests/`	Workloads, NetworkPolicies, reliability, observability, RBAC
`policy/governance/remediation.rego`	Deterministic authorization
`policy/conftest/kubernetes-security.rego`	Manifest policy tests
`python/`	Agent services, shared runtime, tests
`scripts/_ui.sh`	Shared terminal UI; source, do not execute
`scripts/01–15`	Validate through teardown

Path	Responsibility
`terraform/01–25`	GCP foundation; resource addresses unchanged after rename

Dependency chain

Terraform foundation → GKE + Workload Identity + Artifact Registry → Pub/Sub + Firestore + KMS + BigQuery + monitoring → image build (`04` or `13`) → namespaces + SA + RBAC → MCP certificates (`07`) → demo workload + scanners → agents → NetworkPolicies + PodMonitoring + HPA + PDB → smoke tests (`09`–`12`) → signed approval (`11`) → controlled execution review → teardown (`15`) then adopt (`03`) before the next apply.

Best practices applied

- Separation of duty: Gemini recommends; OPA decides; humans sign; MCP executes.
- Fail-closed defaults for automation and reporting.
- Least privilege Workload Identity; no static node keys.
- Immutable signed digests for cluster deploys.
- Evidence that is queryable without putting consumer Gmail on dataset ACLs.

Phase 3: Plan generation

Create a detailed implementation plan as a Markdown document with these eight sections:

1. Overview/Summary
2. File Structure
3. Detailed Implementation Steps (with checkboxes)
4. Dependencies & Prerequisites
5. Potential Issues & Mitigations
6. Testing Strategy
7. Success Criteria
8. Next Steps

`[x]` means implemented in the repository or proven on the live project. `[ ]` means environment-specific or remaining work.

1. Overview / Summary

Complete and validate a portfolio-grade platform that demonstrates responsible agentic AI on Kubernetes. Analysis, authorization, approval, execution, and evidence must stay separate. The model cannot bypass deterministic governance.

2. File structure

```
agentic/
■■■ .github/workflows/
■   ■■■ ci.yml
■   ■■■ release.yml
■■■ docker/                  # one file per runtime image
■■■ docs/
■   ■■■ architecture/        # this Plan Mode record and diagrams
■   ■■■ concepts/
■   ■■■ RUNBOOK.md
```

```
■■■ manifests/              # workloads, NetworkPolicies, HPA, PDB, PodMonitoring
■■■ policy/
■   ■■■ conftest/
■   ■■■ governance/
■■■ python/                 # agents and tests
■■■ scripts/
■   ■■■ _ui.sh
■   ■■■ 01-validate.sh
■   ■■■ 02-conftest-test.sh
■   ■■■ 03-adopt-retained-gcp.sh
■   ■■■ 04-build-images.sh
■   ■■■ 05-install-falco.sh
■   ■■■ 06-create-demo-secret.sh
■   ■■■ 07-generate-mcp-mtls.sh
■   ■■■ 08-deploy.sh
■   ■■■ 09-test-governance.sh
■   ■■■ 10-test-pipeline.sh
■   ■■■ 11-review-approval.sh
■   ■■■ 12-smoke-test-metrics.sh
■   ■■■ 13-release-images.sh
■   ■■■ 14-deploy-release-images.sh
■   ■■■ 15-teardown.sh
■■■ terraform/
    ■■■ 01-variables.tf … 25-outputs.tf
    ■■■ terraform.tfvars.example
```

Notable Terraform names after the rename: `20-observability.tf`, `21-governance-iam.tf`, `22-kms.tf`, `23-firestore-ttl.tf`, `24-evidence.tf`. Resource addresses were not changed; no state migration is required.

3. Detailed implementation steps

Repository hygiene

- [x] Keep only workflows and canonical runtime files.
- [x] Number scripts `01`–`15` and Terraform `01`–`25`.
- [x] Shared terminal UI in `scripts/_ui.sh` (section bars, INFO/OK/WARNING/ERROR/ACTION).
- [x] Replace committed PostgreSQL credentials with generated Secret workflow (`06`). `06` does not rotate an existing Secret.
- [x] Remove retired `cert_guardian_agent` and old Plan Mode binaries from `docs/` root.

Terraform and Google Cloud foundation

- [x] GCS backend `gs://tiqs-kubernetes/agentic/terraform/state`.
- [x] Provision VPC, subnet, GKE, node pool, Artifact Registry, APIs.
- [x] Pub/Sub, Workload Identity, Firestore, KMS, BigQuery, Secret Manager, monitoring.
- [ ] Re-review `terraform plan` after any tfvars CIDR or signer change.
- [x] `03-adopt-retained-gcp.sh` imports KMS key rings, WIF pools, and custom roles that survive destroy.

Build and supply chain

- [x] Eleven images tagged `v1` (including `app` and `scanner-publisher`).
- [ ] Re-run `13-release-images.sh` when a signed digest release is required.
- [ ] Verify Cosign identity when `COSIGN_CERTIFICATE_IDENTITY` is set.

Kubernetes platform

- [x] Seven namespaces in `manifests/namespaces.yaml` plus Helm `falco`.
- [x] Service accounts and least-privilege RBAC.
- [x] MCP CAs and Secrets via `07`.
- [x] MCP Gateway, MCP Server, PostgreSQL, `broken-app`.
- [x] Event Aggregator and agent workloads via `08`.

- [x] NetworkPolicies, HPA, PDB, PodMonitoring, quotas, limits.
- [x] DNS NetworkPolicy paths for Dataplane V2 and NodeLocal DNSCache.

Telemetry and AI analysis

- [x] Falco Helm 9.1.0 (`05`).
- [x] Trivy/Prowler CronJobs. Trivy targets `app`, `mcp-server`, `event-aggregator`, `scanner-publisher` `@v1`. Prowler 5.35.0 is `/home/prowler/.venv/bin/prowler`.
- [ ] Re-run `10-test-pipeline.sh` after each cluster rebuild.

Governance and approval

- [x] `remediation.rego` mounted in `ai-governance` and `ai-agents`.
- [x] `09-test-governance.sh` expects `REQUIRE_APPROVAL` with automation disabled.
- [x] `11-review-approval.sh` signs with KMS version `2`.
- [ ] Keep a recorded screenshot of a signed approve/deny for the portfolio.

Remediation and evidence

- [x] Execution mode disabled during dry-run.
- [x] BigQuery ACL is only `evidence-admin`; impersonate from the Workspace user.
- [ ] One allowlisted controlled action after formal review.

Controlled activation

- [ ] Obtain formal review approval.
- [ ] Set governance and execution modes to `controlled`.
- [ ] Run one reversible action against `app01`.
- [ ] Confirm evidence, then return to `disabled` if continuous automation is not required.

Teardown

- [x] `15-teardown.sh` requires `CONFIRM_TEARDOWN=yes`. Optional `SKIP_*` flags.
- [ ] Next apply after destroy starts with `03-adopt-retained-gcp.sh`.

4. Dependencies and prerequisites

Dependency	Purpose
Terraform >= 1.10	init, validate, plan, apply, destroy
gcloud and kubectl	Auth, Artifact Registry, GKE, Pub/Sub, KMS, Kubernetes
Docker + Buildx	Image build (`04` / `13`)
Python 3.11 + pytest	Agents, tests, secret generation
OPA + Conftest	`01` / `02`
OpenSSL	`07` MCP certificates
Helm	`05` Falco
Trivy, Syft, Cosign	`13` release evidence
Git Bash notes	Pin `CLOUDSDK_PYTHON`; never `bq.cmd query`; `tr -d '\r'` on `gcloud` lists; unset impersonation before kubectl

5. Potential issues and mitigations

Risk	Mitigation
GKE control plane unreachable	Put the workstation `/32` in ignored `terraform.tfvars` and reapply.
VS Code cannot resolve GitHub Actions	Editor resolver issue; validate with GitHub-hosted runs.
Default Firestore database already exists	`manage_firestore_database=false`; manage fields and TTL only.
Artifact Registry destroy blocked	Delete packages (strip `\r` on Git Bash) before `terraform destroy`.
Pub/Sub stage stalls	Consumer logs, IAM, schema, `agent-events-dlq-sub`.
Approval signature rejected	Canonical JSON, request hash, **key version 2**, reviewer, expiry.
NetworkPolicy blocks DNS	kube-dns + node-local-dns pods, `10.112.0.10/32`, `169.254.20.10/32`.
Duplicate execution	Firestore transactions and idempotency keys before MCP.
kubectl as evidence-admin	Unset `CLOUDSDK_AUTH_IMPERSONATE_SERVICE_ACCOUNT`; delete `gke_gcloud_auth_plugin_cache`.
Org policy on BQ/IAM	Do not add Gmail or `projectOwners` to the dataset ACL. Impersonate `evidence-admin`.
`06` does not rotate Postgres	Delete the Secret and postgres pod, then re-run `06`.
Destroy cannot drop BQ tables	Delete the dataset as `evidence-admin` first (`15` does this).

6. Testing strategy

Test layer	Method	Acceptance
Static code	`01-validate.sh` compileall + pytest	Pass
YAML	Parse manifests	Pass
Shell	`bash -n` on `scripts/*.sh` including `_ui.sh`	Pass
Terraform	fmt, init -backend=false, validate	Pass
OPA	`opa test policy/governance -v`	Pass
Conftest	`02-conftest-test.sh`	Active manifests pass; `broken-app.yaml` fails
Supply chain	`13` Trivy / Syft / Cosign	No gated HIGH/CRITICAL; SBOM + signature
Kubernetes	Rollout status in `08` / `14`	All Deployments ready
Metrics	`12-smoke-test-metrics.sh`	Agent metrics present
Governance	`09-test-governance.sh`	`REQUIRE_APPROVAL`, no remediation
Pipeline	`10-test-pipeline.sh`	Incident report received
Approval	`11-review-approval.sh`	Signed decision published

7. Success criteria

- [x] Every agent uses a dedicated identity.
- [x] Event stages have retry and dead-letter handling.
- [x] Gemini cannot authorize or execute Kubernetes actions.
- [x] OPA produces deterministic decisions.
- [x] Critical / policy-selected actions require signed approval.
- [x] Only the Remediation Agent authenticates to the MCP gateway.
- [x] MCP Server RBAC is limited to allowlisted `app01` operations.

- [ ] Recorded Cosign verification for a recruiter demo.
- [ ] Recorded BigQuery evidence listing via impersonation.
- [x] Platform can be torn down with `15` and re-adopted with `03`.

8. Next steps

1. Confirm workstation CIDR and GitHub environment reviewers.
2. `./scripts/01-validate.sh`
3. `./scripts/03-adopt-retained-gcp.sh` if the last destroy left KMS/WIF/roles.
4. `terraform -chdir=terraform plan` then apply.
5. `./scripts/04-build-images.sh` or `13` + `14` for signed digests.
6. `./scripts/05-install-falco.sh` then `./scripts/08-deploy.sh`
7. `09`, `12`, `10`, `11` as documented in `docs/RUNBOOK.md`
8. Formal review before `controlled` mode.
9. Portfolio evidence: logs, plan output, approval payload, BigQuery rows.

Time estimation

From the template's additional Plan Mode features: estimated time, complexity 1–10, prerequisites checklist, and rollback strategy.

Workstream	Estimate
Repository cleanup and local validation	4–8 hours
Terraform review and cloud provisioning	6–12 hours
Image build and release evidence	4–8 hours
Kubernetes deployment and troubleshooting	8–16 hours
Governance, approval, and controlled-action testing	8–16 hours
Documentation and portfolio evidence	4–8 hours
Student-lab total	Approximately 34–68 focused hours

Complexity remains **9/10**.

Prerequisites checklist

- [x] Google Cloud project `class-6-5-tiqs` with required APIs.
- [x] GCS Terraform state bucket.
- [ ] Current workstation public IP `/32` in `terraform.tfvars`.
- [x] Workspace signer `APPROVER_EMAIL` for KMS and BigQuery impersonation.
- [ ] GitHub environment reviewers for digest deploy.
- [x] Local Terraform, gcloud, kubectl, Docker, Python 3.11, Helm, OpenSSL.

Rollback strategy

1. Leave `AUTOMATION_MODE` and `EXECUTION_MODE` disabled.
2. Revert a digest deploy with the previous `image-lock.json` via `14`.
3. Rotate MCP client certificates if the mTLS boundary is suspect.
4. `CONFIRM_TEARDOWN=yes ./scripts/15-teardown.sh` for a full lab reset.

5 Next apply must start with `03-adopt-retained-gcp.sh`.

Security and Compliance

For enterprise-minded labs the template requires security considerations, compliance requirements, audit logging, and access control review.

Control	How this lab meets it
Security considerations	Fail-closed policy, signed approvals, mTLS, NetworkPolicy, least-privilege Workload Identity, immutable digests.
Compliance requirements	Deterministic OPA decisions; no model-authorized cluster change; evidence retained 90 days in BigQuery.
Audit logging	Cloud Logging plus BigQuery dataset `agentic_governance_evidence` owned by `evidence-admin`.
Access control review	Gmail Owner for cluster/Terraform only. Workspace user signs. Dataset ACL excludes consumer Gmail and `projectOwners`.

Multi-Environment Support

When planning, consider development, staging, production, and environment-specific variables.

Environment	This lab
Development / student lab	Live `vertex-agent-lab` in `us-central1-c`. Fail-closed. Local `terraform.tfvars` (gitignored).
Staging	Not provisioned. Same Terraform with a second `tfvars` and cluster name would be the path.
Production	Not in scope. GitHub `production` environment plus `14` digest deploy is the production-shaped gate.
Environment-specific variables	`project_id`, `zone`, `authorized_networks`, `approval_signer_members`, `approval_key_version`, `policy_id`, `manage_firestore_database`. Defaults live in `terraform/terraform.tfvars.example`.

Phase 4: Review

Please review this plan. Is it aligned with your requirements? Should I proceed with implementation?

- [x] Repository contains numbered scripts and Terraform only (no retired Phase 1–4 script names).
- [x] No plaintext secrets committed (`*.tfvars` ignored).
- [ ] Terraform plan reviewed for the current CIDR and signer list.
- [x] Python, YAML, Bash, Terraform, OPA, and Conftest wired through `01` / `02`.
- [ ] Image scan, SBOM, signature verification recorded for the demo tag.
- [ ] All Deployments ready after the latest `08` or `14`.
- [x] Governance dry-run path documented (`09`).
- [x] Cloud KMS version `2` documented.
- [ ] Rollback / teardown rehearsed with `15`.

Approval gate: Reviewer decision: [ ] Approve controlled implementation [ ] Request changes [ ] Reject.

Reviewer: _____ Date: _____ Evidence reference: _____

DO NOT implement anything until the plan is approved.

Phase 5: Implementation

Only after the plan is approved, generate:

- Complete, production-ready code for all files
- Configuration files
- Documentation
- Deployment instructions

Preserve fail-closed defaults. Do not enable controlled remediation without explicit approval.

Deployment sequence

```
chmod +x scripts/*.sh
./scripts/01-validate.sh

./scripts/03-adopt-retained-gcp.sh   # after destroy / 409 on retained names

terraform -chdir=terraform init
terraform -chdir=terraform plan -out=agentic.tfplan
terraform -chdir=terraform apply agentic.tfplan

gcloud container clusters get-credentials vertex-agent-lab \
  --zone us-central1-c \
  --project class-6-5-tiqs

./scripts/04-build-images.sh
./scripts/05-install-falco.sh
./scripts/08-deploy.sh

./scripts/12-smoke-test-metrics.sh
./scripts/09-test-governance.sh
./scripts/10-test-pipeline.sh

./scripts/11-review-approval.sh approve analyst@example.com \
  "Approved after validating the incident evidence and proposed target."

CONFIRM_TEARDOWN=yes ./scripts/15-teardown.sh
```

Scripts print cyan section bars and timestamped INFO/OK/WARNING/ERROR/ACTION lines (`COLOR_OUTPUT=auto|always|never`).

Operational handoff

Handoff item	Operational requirement
Owner	AI Platform / Cloud Security engineering
Primary runbook	`docs/RUNBOOK.md`
Runtime health	Probes, PodMonitoring, Cloud Monitoring, Pub/Sub backlog
Security evidence	Cloud Logging, BigQuery (impersonate evidence-admin), Firestore, `release-evidence/`
Incident path	Agent logs, debug subscriptions, DLQ
Emergency stop	`AUTOMATION_MODE` and `EXECUTION_MODE` disabled; rotate MCP certs if needed
Change control	PR validation, signed release, reviewed digest deploy (`14`)

Appendix A: Major component summary

Component	Role
GKE + Workload Identity	Isolated workloads; no static cloud keys on nodes
Artifact Registry	Immutable `v1` images and digest locks
Pub/Sub	Pipeline stages with retries and DLQ
Event Aggregator	Normalizes Falco, Trivy, Prowler, Logging, SCC
Observer / Correlation / IR Analyst	Validate, join, and Gemini-assist
Governance Agent + OPA	Deterministic authorization
Cloud KMS v2 + Approval Agent	Signed human approval
Remediation Agent	Independent PEP; mTLS client
MCP Gateway / Server	nginx mTLS + scoped Kubernetes tools
Reporting Agent	Incident reports; Slack/Jira off by default
Firestore	Correlation, approval, execution state
BigQuery	Durable evidence (`evidence-admin` OWNER)
Falco / Trivy / Prowler	Runtime, image, and cloud-posture findings
GitHub Actions / WIF	CI and keyless release

Elevator speeches for DevSecOps and executive audiences are in Appendix E.

Appendix B: Complete Plan Mode prompt template

Copy this block, replace the request if needed, and paste it into chat. This is the direct-copy form from the 30 June 2026 template, adapted to this repository.

```
You are acting as Cursor AI in PLAN MODE. I need you to follow this
workflow:

### PHASE 1: CLARIFICATION
First, analyze my request and ask me clarifying questions about:
- Project scope and requirements
- Technologies and stack involved
- Existing codebase and constraints
- Specific goals and success criteria
- Any blockers or dependencies
ONLY ask questions - do NOT write any code yet.

### PHASE 2: RESEARCH
After I answer your questions:
- Review the information I've provided
- Identify relevant files and components needed
- Check for potential dependencies
- Consider best practices and patterns

### PHASE 3: PLAN GENERATION
Create a detailed implementation plan with:
- File structure and organization
- Step-by-step task checklist
- Code references and specific components
```

```
- Dependencies and installation steps
- Potential issues and mitigation strategies
- Testing strategy

Structure the plan as a Markdown document with:
1. Overview/Summary
2. File Structure
3. Detailed Implementation Steps (with checkboxes)
4. Dependencies & Prerequisites
5. Potential Issues & Mitigations
6. Testing Strategy
7. Success Criteria
8. Next Steps

In the plan, include:
- Estimated time for implementation
- Complexity rating (1-10)
- Prerequisites checklist
- Rollback strategy

For enterprise projects, include:
- Security considerations
- Compliance requirements
- Audit logging needs
- Access control review

When planning, consider:
- Development environment
- Staging environment
- Production environment
- Environment-specific variables

### PHASE 4: REVIEW
Ask me to review the plan and provide feedback.

### PHASE 5: IMPLEMENTATION
Only after I approve the plan, generate:
- Complete, production-ready code for all files
- Configuration files
- Documentation
- Deployment instructions
DO NOT implement anything until the plan is approved.

---
### MY REQUEST
[Describe your use case here]

During PLAN MODE:
1. Focus on strategy and design, not implementation details
2. Ask specific, technical questions
3. Reference existing code patterns when relevant
4. Identify potential problems before they occur
5. Provide clear reasoning for each decision
6. Wait for approval before writing any code

Let's begin with PHASE 1: CLARIFICATION.
```

This repository's filled request

MY REQUEST: Maintain the governed agentic security platform on GKE in `class-6-5-tiqs`. Preserve fail-closed defaults, numbered scripts `01`–`15`, Terraform `01`–`25`, KMS approval version 2, and the identity split between Gmail Owner, Workspace signer `APPROVER_EMAIL`, and `evidence-admin`. Do not enable controlled remediation without explicit approval.

Appendix C: How to use the template

Option 1: Direct copy (recommended)

Copy the entire template in Appendix B, replace `[Describe your use case here]` with the actual request, and paste it into the chat.

Option 2: Modular approach

For clarification only

Cursor AI - PLAN MODE: I need to implement [feature description]. Ask me clarifying questions first, then create a detailed implementation plan. No code until I approve the plan.

For plan only

Acting as Cursor AI in PLAN MODE, create a detailed implementation plan for [describe your use case]. Include file structure, step-by-step tasks, dependencies, and potential issues. Do NOT write any code yet.

For full workflow

Acting as Cursor AI in PLAN MODE, follow the complete workflow: 1. Ask clarifying questions. 2. Create an implementation plan. 3. Wait for my approval. 4. Generate the code. For this use case: [describe your use case].

Quick start commands

Scenario	Prompt
New feature	Cursor AI - PLAN MODE: I need to implement [feature]. Ask clarifying questions first, then create a detailed implementation plan. No code until I approve the plan.
Bug fix	Cursor AI - PLAN MODE: I'm fixing [bug]. Ask about current behavior, root cause, and expected outcome. Then plan the fix approach.
Infrastructure	Cursor AI - PLAN MODE: I need to provision [GCP resources]. Ask about region, naming, security requirements, and existing resources. Then create a Terraform implementation plan.
Refactoring	Cursor AI - PLAN MODE: I need to refactor [component]. Ask about the current structure, pain points, and desired architecture. Then create a refactoring plan with step-by-step changes.

Conditional planning

During PLAN MODE:

- If requirements are clear, ask 3–5 clarifying questions and proceed to plan.
- If requirements are vague, ask more detailed questions before planning.
- Suggest alternative approaches if you see a better solution.
- Reference common patterns or best practices.

Tips for better Plan Mode results

1. **Be specific.** Include file names, function names, and technical details.
2. **Provide context.** Share error logs, existing code, or architecture diagrams.
3. **Set boundaries.** Specify what is in and out of scope.
4. **Indicate priority.** Which parts are most important?
5. **Mention constraints.** Time, budget, or technical limitations.
6. **Reference standards.** Any class- or company-specific patterns or guidelines.

This template mimics Cursor AI Plan Mode by structuring the conversation into clear phases, forcing clarification before any code is written, creating detailed plans with checkboxes, requiring approval before implementation, and

including best practices and edge cases.

Appendix D: Lab changes through 18 August 2026

1. **Script numbering.** Operator scripts are `01`–`15`. CI calls `02-conftest-test.sh`. Release calls `13` and `14`. Teardown is `15-teardown.sh`.
2. **Shared UI.** `scripts/_ui.sh` adds section separators and colors. Source it; do not execute it.
3. **Terraform numbering.** Files are `01-variables.tf` through `25-outputs.tf`. State addresses unchanged.
4. **Cluster facts.** Zone `us-central1-c`, `e2-standard-4`, Dataplane V2, image tag `v1`, KMS approval version `2`.
5. **Identity split.** Gmail Owner for cluster/Terraform. Workspace `APPROVER_EMAIL` for signing and BQ impersonation. Dataset OWNER is only `evidence-admin`.
6. **NetworkPolicy DNS.** kube-dns, node-local-dns, ClusterIP `10.112.0.10/32`, NodeLocal DNSCache `169.254.20.10/32`.
7. **Git Bash.** `tr -d '\r'` on package lists; Unix `bq` with `CLOUDSDK_PYTHON`; never `bq.cmd query`.
8. **kubectl cache.** Unset impersonation before kubectl; delete `%USERPROFILE%\.kube\gke_gcloud_auth_plugin_cache` if needed.
9. **Secrets.** `06-create-demo-secret.sh` leaves an existing `postgres-credentials` Secret unchanged.
10. **Architecture docs.** Diagrams match live namespaces. Falco is Helm namespace `falco`. MCP client is Remediation Agent only.
11. **Plan Mode documents.** This summary follows the 30 June 2026 prompt template: five phases, eight-part plan, time/security/environment extras, and a reusable prompt in Appendix B.

Appendix E: Real-world relevance and speaking notes

This appendix is the recruiter-facing close. The lab is a student portfolio, not a managed product. The patterns are the ones enterprises use when they want AI assistance without giving a model the keys to production.

Real-world relevance

Security operations already receive more alerts than analysts can investigate. Teams then face a second pressure: leadership wants AI to help, while risk, legal, and audit refuse to let a model change infrastructure on its own.

This platform maps to that problem:

Enterprise pain	What this lab demonstrates
Alert volume	Falco, Trivy, Prowler, Cloud Logging, and SCC normalize into one Pub/Sub pipeline.
Duplicate and weak signals	Correlation Agent plus Firestore state before Gemini sees the incident.
Analyst time	IR Analyst uses Gemini 2.5 Flash to summarize and recommend only.
Untrusted AI actions	OPA decides `DENY`, `NO_ACTION`, `PERMIT`, or `REQUIRE_APPROVAL`. The model cannot authorize.
Sensitive change control	Cloud KMS version 2 binds a Workspace reviewer to an exact request hash and expiry.
Blast radius	Only `app01` / `broken-app` is allowlisted. MCP RBAC is namespace-scoped.
Identity sprawl	Workload Identity and GitHub OIDC. No long-lived node keys or CI keys.

Enterprise pain	What this lab demonstrates
Audit and replay	Firestore idempotency, Cloud Logging, and BigQuery evidence owned by `evidence-admin`.
Supply chain	Trivy, Syft, Cosign, immutable digests, WIF release.

Those controls are the same categories a regulated shop asks for: least privilege, deterministic policy, human-in-the-loop for high impact, cryptographic approval, network isolation, and durable evidence. The lab is small; the control model is not.

Enterprise application and compensation outlook

Where this work lands in a company

- **DevSecOps / Cloud Security.** Policy as code, Kubernetes NetworkPolicy, Falco/Trivy, signed releases, evidence pipelines.
- **AI Platform / MLOps security.** Agent identities, tool-server (MCP) boundaries, prompt-to-action isolation, fail-closed execution.
- **Platform Engineering.** GKE, Workload Identity, HPA/PDB, GitHub Actions WIF, Artifact Registry.
- **Security architecture / GRC.** Approval custody, org-policy IAM domains, BigQuery retention, reviewer separation.

A hiring manager can map the repo to a production conversation: "How do you let Gemini help an analyst without letting it restart a Deployment?" The answer is this design: analyze, authorize, approve, execute, evidence — five independent hops.

What would still be required before enterprise use

Organization-specific threat models and policies; private GKE; managed certificate rotation; formal key custody; performance and disaster-recovery testing; SLOs and on-call; independent penetration testing; cost and retention governance; production change management. Those gaps are documented in `README.md` Scope and limitations.

Compensation outlook (United States, 2026 market, not an offer)

Ranges below are public 2026 market bands for adjacent roles. Total compensation (base + bonus + equity) sits above base in large-tech and regulated finance. Location, clearance, and years of production ownership move the number more than a lab repo alone.

Role this lab supports	Typical US base (2026)	Senior / staff total compensation	Why this repo is relevant
DevSecOps Engineer	$125,000–$185,000	$185,000–$250,000+	Kubernetes security, OPA, Falco/Trivy, IaC, CI signing
Cloud / Platform Security	$140,000–$195,000	$200,000–$280,000	GKE WI, NetworkPolicy, KMS approvals, evidence
AI Platform Engineer (security-minded)	$150,000–$210,000	$220,000–$320,000	Agent isolation, MCP mTLS, fail-closed tool use
Platform Engineer / SRE	$140,000–$190,000	$190,000–$280,000	Terraform, GKE, HPA/PDB, GitHub WIF
Security Architect (cloud/AI)	$170,000–$230,000	$230,000–$340,000	Control plane design, approval custody, audit

Skills that currently carry a premium on DevSecOps and platform postings: Kubernetes security (CKS-shaped work), policy as code (OPA), container scanning, Terraform, and one production-shaped AI-application security story. This lab is that story if the demo is recorded: governance dry-run, signed approval, evidence query, teardown.

Sources for the bands: 2026 DevSecOps salary guides (KORE1, Neat Stack, and adjacent DevOps/platform benchmarks). Treat them as orientation, not a claim about any employer.

Thirty-second elevator speeches

DevSecOps perspective

I built a governed multi-agent security platform on GKE. Runtime, image, and cloud findings land on Pub/Sub. Agents correlate and Gemini helps the analyst, but Gemini never authorizes. OPA makes the decision, Cloud KMS signs human approval, Firestore blocks replay, and the only path to Kubernetes is an mTLS MCP gateway with namespace-scoped RBAC. Releases are scanned, SBOMed, and Cosign-signed over Workload Identity Federation. Fail-closed is the default: automation and Slack/Jira stay off until a reviewer says otherwise.

Executive leadership perspective

Security teams drown in alerts, and the business wants AI to help without creating a new outage or audit finding. This platform shows how to do both. AI shortens investigation. Policy, signed human approval, and a narrow execution boundary decide what actually changes. Every sensitive action leaves evidence in logs and BigQuery. We can demonstrate the full path — detect, decide, approve, act, prove it — and we can turn automation off instantly. The outcome is faster response with a control story a CISO, auditor, and board can accept.

Final review note

Use this document as the planning and review record. Use `README.md` for the recruiter-facing overview and `docs/RUNBOOK.md` for exact commands. The next technical milestone is a recorded fail-closed demo (governance, approval, evidence, teardown) and a reviewed decision before any `controlled` remediation.